



Bachelor-Thesis

Entwicklung und Evaluation agentenbasierter Systeme im
Software Engineering: Ein Ansatz zur Automatisierung von
Entwicklungsprozessen mittels Large Language Models

zur Erlangung des akademischen Grades

Bachelor of Science

im dualen Studiengang Informatik an der IU Internationale Hochschule

von

Maria Musterfrau

Matrikelnummer: 1234567

Musterstraße 1, 12345 Musterstadt

31. Januar 2026

Referent: Prof. Dr. Klaus Quibeldey-Cirkel

Korreferent: Prof. Dr. Philipp Diebold

Kurzfassung

Kontext und Motivation: Large Language Models (LLMs) haben sich von reinen Textgeneratoren zu agentischen Systemen entwickelt, die selbstständig planen, Werkzeuge nutzen und mehrschrittige Aufgaben bewältigen können. Im Software Engineering eröffnet das neue Möglichkeiten für die Automatisierung komplexer Workflows wie Code-Review, Testgenerierung und Refactoring.

Zielsetzung: Die Arbeit untersucht die Entwicklung und Evaluation agentischer Architekturen für Software-Engineering-Aufgaben. Zentrale Forschungsfragen sind: (1) Wie lassen sich robuste Agentenstrategien für SE-Workflows systematisch modellieren? (2) Welche Architekturprinzipien ermöglichen sichere und nachvollziehbare Werkzeugintegration? (3) Mit welchen Metriken kann die Leistungsfähigkeit agentischer Systeme realistisch bewertet werden?

Methodik: Basierend auf einer systematischen Literaturanalyse wird eine Referenzarchitektur entwickelt, die Planung (ReAct-Pattern), Werkzeugnutzung (Linter, Tests, VCS), episodisches Gedächtnis und Sicherheitsmechanismen kombiniert. Ein prototypisches System wurde in Python implementiert und anhand reproduzierbarer Benchmarks evaluiert.

Ergebnisse: Die Evaluation zeigt, dass die entwickelte Architektur in kontrollierten Szenarien eine Erfolgsrate von 73 % bei automatisiertem Refactoring erreicht, während die durchschnittliche Bearbeitungszeit um 45 % reduziert wird. Reflexionsmechanismen verbessern die Robustheit bei fehlerhaften Werkzeugausgaben um 28 %. Kostenanalysen belegen, dass optimierte Kontextverwaltung die Token-Nutzung um bis zu 40 % senken kann.

Beitrag: Die Arbeit liefert eine praxisnahe Referenzarchitektur, konkrete Implementierungsrichtlinien und evaluierte Metriken für agentische SE-Systeme. Sie zeigt sowohl technische Potenziale als auch kritische Limitierungen auf und diskutiert gesellschaftliche Implikationen der Automatisierung.

Abstract

Context and Motivation: Large Language Models (LLMs) have evolved from pure text generators to agentic systems capable of planning, using tools, and handling multi-step tasks autonomously. In software engineering, this opens up new possibilities for automating complex workflows such as code review, test generation, and refactoring.

Objective: This thesis investigates the development and evaluation of agentic architectures for software engineering tasks. Central research questions are: (1) How can robust agent policies for SE workflows be systematically modeled? (2) Which architectural principles enable secure and traceable tool integration? (3) Which metrics can realistically assess the performance of agentic systems?

Methodology: Based on a systematic literature review, a reference architecture is developed that combines planning (ReAct pattern), tool usage (linters, tests, VCS), episodic memory, and security mechanisms. A prototypical system was implemented in Python and evaluated using reproducible benchmarks.

Results: The evaluation shows that the developed architecture achieves a success rate of 73 % in automated refactoring in controlled scenarios, while reducing average processing time by 45 %. Reflection mechanisms improve robustness against erroneous tool outputs by 28 %. Cost analyses demonstrate that optimized context management can reduce token usage by up to 40 %.

Contribution: The work provides a practical reference architecture, concrete implementation guidelines, and evaluated metrics for agentic SE systems. It highlights both technical potentials and critical limitations and discusses societal implications of automation.

Danksagung

An dieser Stelle möchte ich meinen aufrichtigen Dank aussprechen.

Zuallererst danke ich meinen Betreuern **Prof. Dr. Klaus Quibeldey-Cirkel** und **Prof. Dr. Philipp Diebold** für die fachliche Unterstützung, wertvollen Hinweise und die Möglichkeit, die Arbeit zu verfassen. Ihre offene und konstruktive Kritik hat maßgeblich zur Qualität der Arbeit beigetragen.

Mein besonderer Dank gilt auch meinen Kommilitoninnen und Kommilitonen, die durch fachliche Diskussionen und Feedback meine Gedanken geschärft haben. Ihre Perspektiven haben mir geholfen, verschiedene Aspekte der Thematik tiefergehend zu beleuchten.

Darüber hinaus möchte ich denjenigen danken, die mir während des Schreibprozesses moralische Unterstützung gegeben haben. Ihre Geduld und Ermutigung waren unverzichtbar für den Erfolg der Arbeit.

Abschließend danke ich den Entwicklern und der Open-Source-Community für die Bereitstellung der vielfältigen Werkzeuge und Bibliotheken, ohne die die Arbeit nicht möglich gewesen wäre.

Hinweis zur geschlechtergerechten Sprache

In der Arbeit wird auf eine geschlechtergerechte Sprache geachtet. Aus Gründen der besseren Lesbarkeit wird in der Arbeit eine Mischung verschiedener Formen geschlechtergerechter Sprache verwendet:

- **Geschlechtsneutrale Formulierungen** (z. B. Studierende, Mitarbeitende, Person)
- **Paarformen** (z. B. Entwicklerinnen und Entwickler, Nutzerinnen und Nutzer)
- **Genderstern** (z. B. Benutzer*innen) zur Sichtbarmachung aller Geschlechter

Alle verwendeten Personenbezeichnungen gelten gleichermaßen für alle Geschlechter. Die Arbeit orientiert sich an den Empfehlungen des Leitfadens „Sag’s doch gleich! Geschlechtergerechte Sprache an Thüringer Hochschulen“ des Thüringer Kompetenznetzwerks Gleichstellung (TKG).

Inhaltsverzeichnis

Kurzfassung	i
Abstract	iii
Danksagung	v
Hinweis zur geschlechtergerechten Sprache	vii
Abbildungsverzeichnis	xiii
Tabellenverzeichnis	xv
Listings	xvii
Abkürzungsverzeichnis	xix
1 Einleitung	1
1.1 Forschungsfragen	2
2 Theoretische Fundierung	5
2.1 Begriffliche Grundlagen: Monitoring und Observability	5
2.2 Kernbausteine der Observability	6
2.3 Die Four Golden Signals nach Google SRE	7
2.4 Die Plattform frag.jetzt und ihre Systemarchitektur	8
2.5 Werkzeuge für Monitoring und Observability	9
3 Methodik und Forschungsdesign	11
3.1 Schritt 1: Service-spezifische Konkretisierung der Four Golden Signals	11
3.2 Schritt 2: Vergleichende Evaluation von Observability-Stack-Kombinationen	11
3.3 Schritt 3: Instrumentierung mit OpenTelemetry	12
3.4 Schritt 4: Rollenbasierte Dashboards	12
3.5 Schritt 5: Alerting-Framework mit Fokus auf False-Positive-Reduktion	12
3.6 Schritt 6: Entwicklung operativer Runbooks	13

3.7	Ergebnisartefakte	13
3.8	Geplante Gliederung (konzeptionell)	13
4	Implementierung und Ergebnisse	17
4.1	Implementierungsdetails	17
4.1.1	Technologiestack	17
4.1.2	Projektstruktur	17
4.1.3	Kernimplementierung: Agent-Controller	18
4.2	Experimentelles Setup	21
4.2.1	Testumgebung	21
4.2.2	Benchmark-Szenarien	21
4.2.3	Beispiel: Test Failure Diagnosis — Schritt-für-Schritt	22
4.3	Ergebnisse	23
4.3.1	Erfolgsraten nach Aufgabentyp	23
4.3.2	Effizienzmetriken	24
4.3.3	Qualitätsmetriken	24
4.3.4	Robustheit und Fehlerbehandlung	24
4.4	Vergleich mit existierenden Ansätzen	25
4.5	Validierung und Verifikation	26
4.5.1	Unit-Tests	26
4.5.2	Integrationstests	26
4.5.3	Sicherheits-Audits	26
5	Fazit und Ausblick	29
5.1	Zusammenfassung der Ergebnisse	29
5.1.1	Beantwortung der Forschungsfragen	29
5.1.2	Empirische Validierung	30
5.2	Beiträge der Arbeit	31
5.2.1	Wissenschaftlicher Beitrag	32
5.3	Limitierungen	32
5.3.1	Methodische Limitierungen	32
5.3.2	Technische Limitierungen	33
5.3.3	Gesellschaftliche und ethische Limitierungen	33
5.4	Zukünftige Arbeiten	34
5.4.1	Kurzfristige Erweiterungen	34

5.4.2 Mittel- bis langfristige Forschungsrichtungen	34
5.4.3 Offene Forschungsfragen	35
5.5 Schlusswort	35
Literaturverzeichnis	37
Index	39
A Verzeichnis der KI-Nutzung	41

Abbildungsverzeichnis

Tabellenverzeichnis

4.1	Vergleich mit Baseline-Systemen (auf gleichem Benchmark-Set) . .	25
4.2	Detaillierte Benchmark-Ergebnisse: Agentische SE-Tasks mit Evaluationsmetriken	27
4.3	Evaluationsmetriken für agentische SE-Workflows (Beispieltabelle) .	27
A.1	Verzeichnis der in dieser Arbeit genutzten KI-Werkzeuge	42

Listings

4.1	Minimaler agentischer Loop für SE-Aufgaben (Beispiel-Listing) . . .	18
4.2	Werkzeugaufruf-Stub in TypeScript mit einfachem Funktionsschema (Beispiel-Listing)	19

Abkürzungsverzeichnis

Abkürzung	Bedeutung
API	Application Programming Interface
CI/CD	Continuous Integration / Continuous Deployment
CPU	Central Processing Unit
ELK	Elasticsearch, Logstash, Kibana
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
KI	Künstliche Intelligenz
NLP	Natural Language Processing
OTel	OpenTelemetry
PWA	Progressive Web App
Q&A	Question and Answer
REST	Representational State Transfer
SLO	Service Level Objective
SPA	Single-Page Application
SRE	Site Reliability Engineering
UI	User Interface

1 Einleitung

Webbasierte Anwendungssysteme werden heute häufig als verteilte, cloud-native Anwendungen umgesetzt, die aus mehreren lose gekoppelten Services bestehen. Diese Architekturform erlaubt eine unabhängige Weiterentwicklung einzelner Komponenten sowie eine flexible Skalierung, erhöht jedoch zugleich den Aufwand für Betrieb, Überwachung und Fehleranalyse im Produktivsystem [Fas+25; Sak25]. Insbesondere bei Microservices entstehen verteilte Fehlerzustände und komplexe Abhängigkeiten zwischen Diensten, wodurch Ursachen von Störungen nicht mehr eindeutig einzelnen Komponenten zugeordnet werden können [Fis+24].

Vor diesem Hintergrund gewinnt Observability als Erweiterung klassischer Monitoring-Konzepte an Bedeutung. Ziel von Observability ist es, den internen Zustand eines Systems anhand extern erfassbarer Anzeichen nachvollziehen zu können. Dazu werden Metriken, Logs und Traces kombiniert, um Zusammenhänge zwischen Systemverhalten, Fehlern und Leistungsmerkmalen sichtbar zu machen [Sri18; Fas+25]. Empirische Untersuchungen aus dem Bereich cloud-nativer Anwendungen zeigen, dass ein strukturierter Einsatz von Observability-Praktiken die Analyse von Incidents unterstützt und die Reaktionszeit im Betrieb verkürzen kann [Gia+24; Sak25].

Ein häufig verwendeter Orientierungsrahmen zur Strukturierung servicebezogener Beobachtungsdaten sind die von Google im Kontext des Site Reliability Engineering (SRE) beschriebenen *Four Golden Signals*: Latenz, Traffic, Fehler und Sättigung [Ewa17]. Diese Signalarten erfassen grundlegende technische Symptome eines Dienstes und eignen sich als Ausgangspunkt zur Ableitung erster Monitoring-Indikatoren. Für den produktiven Betrieb liefern sie jedoch primär Rohinformationen, die erst durch Kontext, Interpretation und Bewertung für Menschen handlungsrelevant werden. Google SRE weist selbst darauf hin, dass die Wirksamkeit solcher Signale maßgeblich davon abhängt, wie sie in Alerting, Entscheidungsprozesse und betriebliche Abläufe eingebettet sind [Ewa17].

Die Open-Source-Plattform *frag.jetzt* verfügt über eine solide Entwicklungs- und Deployment-Infrastruktur, jedoch ist die Erfassung und Auswertung von Betriebs-

daten derzeit nicht als durchgängiges Observability-Konzept ausgelegt. Metriken, Logs, Traces und Alerting werden nicht konsistent entlang des gesamten Anfragepfads betrachtet, wodurch Zusammenhänge zwischen Nutzerverhalten, Backend-Verarbeitung und externen Abhängigkeiten nur eingeschränkt nachvollziehbar sind. Diese Einschätzung stützt sich auf die öffentlich verfügbare Projektdokumentation und Architekturübersicht des *frag.jetzt*-Repositories [Ta25] sowie auf allgemeine Beobachtungen zu typischen Defiziten in vergleichbaren Systemen [Gia+24].

Ziel dieser Arbeit ist die Entwicklung einer Observability-Strategie für *frag.jetzt*, bei der technische Messgrößen systematisch mit nutzungsbezogenen Fragestellungen verknüpft werden. Die Four Golden Signals dienen dabei als unterstützender Referenzrahmen zur Strukturierung der Beobachtungsdaten, nicht jedoch als alleinige Grundlage für betriebliche Entscheidungen. Die Signale werden servicespezifisch für das PWA-Frontend, das Spring-Boot-Backend, die PostgreSQL-Datenbank sowie angebundene KI-Dienste konkretisiert und in messbare Indikatoren überführt. Ergänzend wird untersucht, welche Open-Source-Observability-Werkzeuge die Anforderungen von *frag.jetzt* im Hinblick auf Erfassung und betriebliche Nutzbarkeit am besten erfüllen.

Ein weiterer Schwerpunkt der Arbeit liegt auf der Ausgestaltung eines Alerting-Ansatzes, der Auffälligkeiten innerhalb der Plattform mit Fragen der Relevanz und Dringlichkeit verknüpft. Dabei steht nicht allein die Detektion von Abweichungen im Vordergrund, sondern insbesondere die Bewertung, ob eine Handlung erforderlich ist, wen ein Problem betrifft und welche Auswirkungen es auf die Nutzung der Plattform hat. Forschungsarbeiten zeigen, dass eine Kombination aus priorisiertem Alerting und datenbasierten Analyseverfahren dazu beitragen kann, irrelevante Alarmierungen zu reduzieren und menschliche Entscheidungsprozesse im Betrieb zu unterstützen [Jal+24]. Aufbauend auf den definierten Alerts werden Runbooks konzipiert, die strukturierte Diagnose- und Handlungsschritte für wiederkehrende Incident-Szenarien bereitstellen.

1.1 Forschungsfragen

Die Forschungsfragen dieser Arbeit lauten:

1. Wie können die Four Golden Signals für die spezifischen Services von *frag.jetzt* (PWA-Frontend, Spring Boot Backend, PostgreSQL, KI-Services) konkret definiert und instrumentiert werden?
2. Welche Observability-Stack-Kombination (Prometheus/Grafana, ELK Stack, Jaeger) ist am besten geeignet für die Monitoring-Anforderungen von *frag.jetzt*?
3. Wie kann ein intelligentes Alert-System entwickelt werden, das False Positives minimiert und actionable insights für verschiedene Stakeholder-Rollen (DevOps, Entwickler, Product Owner) bereitstellt?

2 Theoretische Fundierung

Moderne Softwarelösungen sind zunehmend durch verteilte Architekturen, Cloud-native Technologien und eine hohe Änderungsdynamik geprägt. Insbesondere Microservice-basierte Anwendungen mit Container-Orchestrierung und Continuous Deployment erhöhen die Komplexität des Betriebs erheblich [Sak25]. Klassische Monitoring-Ansätze, die primär auf vordefinierte Metriken und statische Schwellwerte setzen, stoßen in solchen Umgebungen an ihre Grenzen, da sie häufig nur bekannte Fehlerzustände erfassen und wenig Unterstützung bei der Ursachenanalyse bieten [Sri18].

Vor diesem Hintergrund hat sich der Ansatz der *Observability* als wichtiges Konzept für den Betrieb moderner Systeme etabliert. Observability zielt darauf ab, den internen Zustand eines Systems anhand externer Messwerte nachvollziehbar zu machen und dadurch auch unbekannte oder unerwartete Fehlerszenarien analysierbar zu gestalten [Abi+25].

2.1 Begriffliche Grundlagen: Monitoring und Observability

Der Begriff Observability stammt ursprünglich aus der System- und Regelungstheorie und beschreibt die Fähigkeit, interne Zustände eines Systems aus dessen Ausgaben abzuleiten. Übertragen auf heutige Softwareanwendungen bezeichnet Observability die Eigenschaft, das Verhalten einer Application durch geeignete Telemetriedaten nachvollziehbar zu machen [Sri18].

Monitoring und Observability sind dabei eng miteinander verbunden, jedoch nicht gleichzusetzen. Monitoring fokussiert sich auf die Überwachung bekannter Metriken und Ereignisse, um Abweichungen vom erwarteten Normalzustand zu erkennen. Observability erweitert diesen Ansatz, indem sie Kontextinformationen bereitstellt, die eine detaillierte Analyse von Ursachen und Zusammenhängen ermöglichen [Fas+25]. In der Literatur wird Observability daher häufig als Weiterentwicklung oder Übermenge des klassischen Monitorings beschrieben.

Zentral für Observability ist die systematische Erfassung unterschiedlicher Telemetriearten. Erst durch die kombinierte Betrachtung verschiedener Daten wird es möglich, komplexe Systemzustände sowie Ursachen von Fehlern zuverlässig zu analysieren [Abi+25; Sak25].

2.2 Kernbausteine der Observability

In der Literatur werden vier Hauptelemente der Observability unterschieden: Metriken, Logs, Traces und Alerting. Diese Bausteine ergänzen sich gegenseitig und bilden gemeinsam die Grundlage für eine umfassende Analyse des Laufzeitverhaltens moderner Softwaresysteme [Abi+25; Sak25].

Metriken sind numerische Messwerte, die den Zustand oder das Verhalten einer Anwendung über die Zeit hinweg beschreiben, beispielsweise Antwortzeiten, Fehlerraten oder Ressourcenauslastung. Sie eignen sich besonders für Trendanalysen, Kapazitätsplanung und die Definition von Service-Level-Zielen. Aufgrund ihrer aggregierten Form ermöglichen Metriken eine effiziente Überwachung auch großer Softwaresysteme [Sak25].

Logs bestehen aus ereignisbasierten Textinformationen, die detaillierte Einblicke in den Ablauf einzelner Systemkomponenten liefern. Sie enthalten kontextreiche Informationen, etwa Fehlermeldungen oder Zustandsänderungen, und sind insbesondere für die Ursachenanalyse und das Debugging grundlegend. Der Nachteil von Logs liegt in ihrem hohen Volumen und der vergleichsweise aufwändigen Auswertung [Abi+25].

Traces dienen der Nachverfolgung einzelner Anfragen über mehrere Services hinweg. Sie zeigen, welche Komponenten an der Verarbeitung beteiligt waren und wie viel Zeit in den jeweiligen Verarbeitungsschritten verbracht wurde. Distributed Tracing ist insbesondere in Microservice-Architekturen ein essenzielles Mittel zur Identifikation von Latenzengpässen und Abhängigkeitsproblemen [AC25].

Alerting ergänzt die passive Beobachtung durch aktive Benachrichtigung bei kritischen Zuständen. Alerts werden typischerweise auf Basis von Metriken oder Log-Ereignissen ausgelöst und sollen Betreiber frühzeitig auf Probleme hinweisen. Eine

zentrale Herausforderung besteht darin, Alarmregeln so zu gestalten, dass relevante Vorfälle erkannt werden, ohne eine hohe Anzahl an Fehlalarmen zu erzeugen [Jal+24].

Erst das Zusammenspiel dieser vier Bausteine ermöglicht eine umfassende Observability. Während Metriken einen schnellen Überblick liefern, stellen Logs und Traces den notwendigen Detailgrad für die Ursachenanalyse bereit. Alerting sorgt schließlich dafür, dass Abweichungen zeitnah erkannt und adressiert werden können.

2.3 Die Four Golden Signals nach Google SRE

Die *Four Golden Signals* stellen ein technisches Referenzmodell zur Beobachtung von Diensten dar und wurden im Rahmen des Site Reliability Engineering von Google entwickelt [Ewa17]. Sie beschreiben vier wesentliche technische Messgrößen, anhand derer sich Symptome von Leistungs- oder Stabilitätsproblemen identifizieren lassen:

- **Latency:** Die Zeitspanne zur Bearbeitung einer Anfrage. Hohe oder stark schwankende Latenzen weisen auf Performanceprobleme hin.
- **Traffic:** Das Anfrage- oder Datenvolumen, das auf einen Dienst einwirkt, beispielsweise gemessen in Requests pro Sekunde.
- **Error:** Die Rate fehlgeschlagener Anfragen, etwa durch Serverfehler oder funktionale Fehlzustände.
- **Saturation:** Der Auslastungsgrad kritischer Ressourcen wie CPU, Speicher oder Warteschlangen.

Google empfiehlt, diese Signale als erste Orientierung im Monitoring zu nutzen, betont jedoch zugleich, dass ihre Aussagekraft stark vom jeweiligen Nutzungskontext abhängt [Ewa17]. Insbesondere für Alerting und operative Entscheidungen ist eine Ergänzung durch Kontextinformationen erforderlich, etwa zur aktuellen Nutzungssituation, zur betroffenen Nutzergruppe oder zur zeitlichen Relevanz eines Problems. Ohne diese Einbettung besteht die Gefahr, dass technisch korrekte Anzeichen zu unnötigen oder nicht handlungsrelevanten Alarmierungen führen.

2.4 Die Plattform frag.jetzt und ihre Systemarchitektur

Die vorliegende Arbeit bezieht sich exemplarisch auf die Q&A-Plattform *frag.jetzt*, eine Cloud-native Webanwendung mit serviceorientierter Architektur. Die Plattform wurde an der Technischen Hochschule Mittelhessen entwickelt und dient der Durchführung interaktiver Fragerunden. Neben klassischer Frage-Antwort-Funktionalität integriert *frag.jetzt* auch KI-gestützte Assistenzfunktionen zur automatisierten Beantwortung von Nutzeranfragen [Ta25].

Zur Einordnung der späteren Observability-Strategie wird im Folgenden ein Überblick über die Architekturkomponenten der Plattform gegeben [Ta25]:

- **Angular Frontend (SPA/PWA):** Eine Single-Page-Application, über die Nutzer mit dem System interagieren. Das Frontend ist als Progressive Web App umgesetzt und kommuniziert überwiegend über HTTPS-APIs sowie WebSockets mit dem Backend.
- **WebSocket-Gateway (NGINX):** Ein Gateway-Server zur Vermittlung der bidirektionalen Echtzeitkommunikation zwischen Frontend und Backend, insbesondere für das asynchrone Pushen von Antworten und Statusupdates.
- **Spring Boot Backend:** Der zentrale, Java-basierte Anwendungsserver, der die Kernlogik der Plattform implementiert. Er verarbeitet Nutzeranfragen, koordiniert interne Dienste und stellt REST- sowie WebSocket-Schnittstellen bereit.
- **Keycloak Identity Service:** Ein externer Dienst zur Authentifizierung und Autorisierung von Nutzern. Er übernimmt das Identitätsmanagement, führt jedoch zusätzliche Abhängigkeiten und potenzielle Latenzpunkte ein.
- **PostgreSQL-Datenbank:** Eine relationale Datenbank zur Persistierung des Anwendungszustands, etwa für Benutzerkonten, Fragen und Antworten. Aufgrund ihrer Rolle bei der Verarbeitung und Speicherung aller zustandsbehafteten Daten stellt sie einen kritischen Ausfallpunkt, einen sogenannten Single Point of Failure, bei hoher Last dar.
- **KI-Backend (Langchain & FastAPI):** Ein eigenständiger Microservice, der über Python (FastAPI) und die Langchain-Bibliothek Anfragen an externe Large-Language-Modelle weiterleitet, um KI-generierte Antworten zu erzeugen. Die Abhängigkeit von externen KI-APIs bringt variable Antwortzeiten und zusätzliche Fehlerquellen mit sich.

- **RabbitMQ Message Broker:** Ein Message-Broker zur asynchronen Entkopplung von Backend- und KI-Komponenten. RabbitMQ ermöglicht das Puffern und Weiterleiten von Antwortnachrichten und unterstützt so eine entkoppelte und skalierbare Verarbeitung.

Diese verteilten Komponenten bilden gemeinsam den durchgängigen Anfragepfad der Plattform. Eine Nutzerfrage durchläuft typischerweise mehrere Dienste, bevor eine Antwort an das Frontend zurückgegeben wird. Dadurch entstehen zahlreiche Schnittstellen und potenzielle Fehlerquellen. Eine Observability-Strategie für *frag.jetzt* muss daher sicherstellen, dass entlang dieses gesamten Pfades geeignete Metriken, Logs und Traces erfasst werden, um Performanceprobleme und Ausfälle nachvollziehbar analysieren zu können [Fas+25].

2.5 Werkzeuge für Monitoring und Observability

Zur Umsetzung von Monitoring- und Observability-Konzepten steht eine Vielzahl spezialisierter Open-Source-Werkzeuge zur Verfügung, die in modernen Cloud- und Microservice-Architekturen etabliert sind. Diese Werkzeuge adressieren unterschiedliche Aspekte der Telemetrieerfassung und -auswertung und werden häufig in Kombination eingesetzt [BS24].

- **Prometheus:** Eine Open-Source-Software zur Erfassung und Speicherung zeitbasierter Metriken. Prometheus arbeitet mit einem Pull-basierten Ansatz, bei dem Metrik-Endpunkte regelmäßig abgefragt werden. Die integrierte Abfragesprache PromQL erlaubt detaillierte Analysen von Zeitreihendaten und bildet die Grundlage für regelbasierte Alerting-Mechanismen. Prometheus ist als CNCF-Projekt weit verbreitet und insbesondere für dynamische Cloud-Umgebungen geeignet [BS24].
- **Grafana:** Eine Plattform zur Visualisierung und Analyse von Observability-Daten aus unterschiedlichen Datenquellen. Grafana dient häufig als Dashboard-Werkzeug, in dem Metriken, Logs und Traces gemeinsam dargestellt und korreliert werden können. Dadurch unterstützt Grafana die visuelle Analyse von Systemzuständen und Trends [Gia+24].
- **Elastic Stack (ELK):** Der Elastic Stack kombiniert Elasticsearch, Logstash und Kibana und wird vor allem für die zentrale Aggregation und Analyse von

Logdaten eingesetzt. Die Stärke des Stacks liegt in der skalierbaren Speicherung unstrukturierter Daten sowie in leistungsfähigen Such- und Analysefunktionen. Neben Logs unterstützt Elastic auch Metriken und Tracing, erfordert jedoch vergleichsweise hohe Systemressourcen [BS24].

- **Jaeger:** Ein verteiltes Tracing-Tool zur Nachverfolgung von Anfragen über Service-Grenzen hinweg. Jaeger ermöglicht die Analyse von End-to-End-Latenzen und unterstützt die Identifikation von Performanceengpässen sowie Abhängigkeitsbeziehungen in Microservice-Architekturen [AC25].
- **OpenTelemetry:** Ein herstellerneutraler Standard zur Instrumentierung von Anwendungen. OpenTelemetry (OTel) stellt ein einheitliches API und SDK zur Erfassung von Metriken, Logs und Traces bereit und erlaubt die Weiterleitung dieser Daten an unterschiedliche Backends. Dadurch wird die Telemetrieerfassung von der Wahl konkreter Observability-Werkzeuge entkoppelt [AC25].

Die Auswahl geeigneter Werkzeuge erfolgt typischerweise anhand mehrerer Bewertungskriterien, darunter die unterstützten Telemetriearten, Integrationsfähigkeit, Skalierbarkeit, Analyse- und Visualisierungsfunktionen sowie der Reifegrad der jeweiligen Lösung [Gia+24]. Diese Kriterien bilden die Grundlage für eine spätere vergleichende Analyse im Rahmen der Methodik.

3 Methodik und Forschungsdesign

Hinweis: Vorläufiger Inhalt

Die folgenden Abschnitte zur Methodik und zur geplanten Gliederung stammen aus dem Exposé und werden im Laufe der Bachelorarbeit überarbeitet, konkretisiert oder angepasst. Sie dienen derzeit als Arbeitsgrundlage und spiegeln den konzeptionellen Stand zu Beginn der Arbeit wider.

Die Arbeit folgt einem artefaktorientierten Forschungsdesign mit konzeptionellem und prototypischem Schwerpunkt. Ziel ist es, eine praxisnahe Observability-Strategie für *frag.jetzt* zu entwickeln und diese anhand konkreter Artefakte nachvollziehbar darzustellen. Die Methodik gliedert sich in sechs aufeinander aufbauende Schritte.

3.1 Schritt 1: Service-spezifische Konkretisierung der Four Golden Signals

Im ersten Schritt werden die Four Golden Signals als Referenzmodell herangezogen und für die Zielarchitektur von *frag.jetzt* konkretisiert. Für jedes Teilsystem (PWA-Frontend, Spring-Boot-Backend, PostgreSQL-Datenbank und KI-Dienste) werden geeignete Interpretationen für die einzelnen Beobachtungsgrößen abgeleitet, beispielsweise Latenz aus Nutzersicht, Fehlerraten pro Service oder Ressourcenauslastung kritischer Komponenten. Ziel ist die Definition klar interpretierbarer Indikatoren, die sowohl für Dashboards als auch für Alerting nutzbar sind.

3.2 Schritt 2: Vergleichende Evaluation von Observability-Stack-Kombinationen

Der zweite Schritt umfasst eine vergleichende Evaluation geeigneter Observability-Werkzeuge. Da eine vollständige praktische Erprobung aller verfügbaren Tools im

Rahmen der Arbeit nicht realistisch ist, erfolgt der Vergleich primär auf theoretischer und konzeptioneller Ebene. Untersucht werden Prometheus/Grafana, der Elastic Stack sowie Jaeger unter Berücksichtigung der Anforderungen von *frag.jetzt*. Bewertungskriterien sind unter anderem die Abdeckung der benötigten Telemetriearten, die Integrationsfähigkeit in bestehende Systeme, der Betriebsaufwand sowie die Verfügbarkeit als Open-Source-Lösung. Auf dieser Basis wird eine begründete Auswahl eines geeigneten Stacks getroffen.

3.3 Schritt 3: Instrumentierung mit OpenTelemetry

Im dritten Schritt wird die Instrumentierung der Services konzipiert. OpenTelemetry dient dabei als einheitliches Instrumentierungswerkzeug zur Erfassung von Metriken, Logs und Traces. Ziel ist eine konsistente Telemetrie entlang des gesamten Anfragepfads, die eine Korrelation zwischen Nutzerinteraktionen, Backend-Verarbeitung und externen Abhängigkeiten ermöglicht.

3.4 Schritt 4: Rollenbasierte Dashboards

Der vierte Schritt befasst sich mit der Konzeption rollenbasierter Dashboards. Für unterschiedliche Zielgruppen wie DevOps, Entwickler und Product Owner werden Sichten entworfen, die Telemetriedaten mit nutzungs- und betriebsrelevantem Kontext verknüpfen. Ziel ist es, nicht nur Auffälligkeiten darzustellen, sondern deren Bedeutung für die jeweilige Rolle nachvollziehbar zu machen, etwa im Hinblick auf Nutzerwahrnehmung, Dringlichkeit oder notwendige Maßnahmen.

3.5 Schritt 5: Alerting-Framework mit Fokus auf False-Positive-Reduktion

Im fünften Schritt wird ein Alerting-Ansatz entwickelt, der sich an den Golden Signals orientiert und auf Prometheus als technischer Basis aufsetzt. Neben klassischen regelbasierten Alerts werden Konzepte zur Priorisierung und Reduktion irrelevanter Alarme berücksichtigt. Dazu gehören beispielsweise die Aggregation

mehrerer Signale, zeitliche Glättung sowie der Einsatz datengetriebener Verfahren wie einfacher Anomalieerkennung. Optional wird untersucht, inwiefern KI-gestützte Verfahren zur Mustererkennung oder Alert-Bewertung eingesetzt werden können, um die Anzahl unnötiger Benachrichtigungen zu reduzieren.

3.6 Schritt 6: Entwicklung operativer Runbooks

Im sechsten Schritt werden exemplarische Runbooks für wiederkehrende Incident-Szenarien erstellt. Diese Runbooks sollen Alerts mit klaren Diagnose- und Handlungsanweisungen verknüpfen und dienen dazu, Reaktionszeiten zu verkürzen und die operative Qualität der Störungsbearbeitung zu erhöhen.

3.7 Ergebnisartefakte

Aus den beschriebenen Schritten ergeben sich die geplanten Ergebnisse:

1. Ein wissenschaftlicher Bericht zur Strategie, Toolauswahl und Alerting-Konzeption
2. Eine lauffähige Observability-Plattform (z. B. Prometheus/Grafana)
3. Rollenbasierte Dashboards
4. Beispielhafte Runbooks für Incident-Situationen

3.8 Geplante Gliederung (konzeptionell)

Hinweis: Vorläufige Gliederung

Die folgende Gliederung ist ein erster Entwurf aus dem Exposé. Die endgültige Struktur der Arbeit kann hiervon abweichen und wird im Verlauf der Bearbeitung angepasst.

1. Einleitung

Problemstellung, Motivation, Zielsetzung und Forschungsfragen; Einordnung der Arbeit in den Kontext cloud-nativer Systeme und moderner Observability.

2. Theoretische Fundierung

Grundlagen zu Monitoring und Observability (Metriken, Logs, Traces, Alerting); Four Golden Signals als Referenzmodell; Überblick relevanter Observability-Tools und Toolklassen.

3. Untersuchungsgegenstand: Architektur von *frag.jetzt*

Beschreibung der Systemarchitektur (PWA-Frontend, Spring Boot Backend, PostgreSQL, KI-Services sowie Schnittstellen/Kommunikationspfade); Ableitung der Observability-Anforderungen je Komponente.

4. Methodik / Forschungsdesign

Vorgehensmodell der Arbeit; Operationalisierung der Golden Signals; Kriterienkatalog für Tool-/Stack-Evaluation; Instrumentierungs- und Evaluationsplan; Validitäts- und Abgrenzungsaspekte.

5. Evaluation von Observability-Stacks

Vergleich Prometheus/Grafana, ELK Stack, Jaeger (und OpenTelemetry als Instrumentierungsstandard); Ergebnisdarstellung anhand definierter Kriterien; begründete Auswahl einer geeigneten Stack-Kombination.

6. Konzeption der Observability-Strategie für *frag.jetzt*

Service-spezifische Golden-Signal-Definitionen; Messkonzept (Metriken/Logs/Traces) entlang des Anfragepfads; Telemetrie- und Korrelationserfordernisse.

7. Umsetzung: Instrumentierung und Observability-Plattform

Umsetzung der Telemetrie-Erfassung; Konfiguration der Plattformkomponenten; Nachweis der End-to-End-Sicht (Metrik-Trace-Log-Korrelation).

8. Dashboards und Alerting-Konzept

Rollenbasierte Dashboards; Alerting-Regeln auf Grundlage nutzerrelevanter Beobachtungsgrößen; Ansatz zur False-Positive-Reduktion und Priorisierung; Einbindung von Anomalieerkennung (konzeptionell/prototypisch).

9. Runbooks und Incident-Playbooks

Erstellung exemplarischer Runbooks; Verknüpfung von Alerts mit Diagnose- und Handlungsschritten; typische Incident-Szenarien.

10. Diskussion

Einordnung der Ergebnisse, Grenzen der Umsetzung, Trade-offs (z. B. Aufwand vs. Nutzen, Datenvolumen/Kardinalität, Abhängigkeit externer KI-Services).

11. Fazit und Ausblick

Zusammenfassung der wichtigsten Ergebnisse; Ausblick auf Weiterentwicklung

(z. B. Reifegradsteigerung, Service Level Objective (SLO)-basierte Steuerung, weitere Automatisierung).

4 Implementierung und Ergebnisse

4.1 Implementierungsdetails

Das Kapitel dokumentiert die praktischen Aspekte der Umsetzung.¹ Die Implementierung erfolgte iterativ über einen Zeitraum von 4 Monaten mit kontinuierlichem Testen und Verfeinerung.

4.1.1 Technologiestack

Folgende Technologien wurden für die Implementierung eingesetzt:

Programmiersprache: Python 3.11+ (Typ-Hinweise, Async/Await-Unterstützung)

LLM-Integration: OpenAI API (GPT-4), Anthropic API (Claude 3.5 Sonnet), mit Fallback-Mechanismus

Orchestrierung: LangChain 0.1.x für Basis-Komponenten, benutzerdefinierte Erweiterungen für SE-spezifische Tools; strukturierte Tool-Calls über **JSON**-Schemas und Konfigurationen per **YAML**.

Gedächtnis: Chroma Vektordatenbank für semantisches Retrieval, SQLite für episodische Logs

Werkzeuglaufzeitumgebung: Docker 24.0+ für Sandboxing, pytest für Tests, ruff/black für Linting

Monitoring: Prometheus für Metriken, strukturierte Logs (JSON) mit Trace-IDs

4.1.2 Projektstruktur

Das Projekt folgt einer modularen Architektur:

```
1 agent_se/  
2 +-- core/  
3 |   +-- agent.py           # Agent Controller (ReAct-Loop)
```

¹ Implementierungsdetails orientieren sich an Best Practices aus [martin2008clean].

```
4 |   +-- policy.py           # Planning & Reflection Logic
5 |   +-- state.py           # State Management
6 +-- tools/
7 |   +-- base.py           # Tool Interface & Registry
8 |   +-- code_tools.py     # Linter, Formatter, AST-Parser
9 |   +-- test_tools.py     # pytest, coverage, Test-Runner
10 |  +-- vcs_tools.py       # git operations
11 +-- memory/
12 |   +-- episodic.py      # Event Logging & Retrieval
13 |   +-- semantic.py      # Vector Store Integration
14 +-- safety/
15 |   +-- validator.py     # Input Validation & Sanitization
16 |   +-- sandbox.py       # Docker-based Execution Env
17 +-- evaluation/
18     +-- benchmarks.py    # Test Scenarios
19     +-- metrics.py       # Success Rate, Costs, Latency
```

4.1.3 Kernimplementierung: Agent-Controller

Listing 4.1 zeigt eine minimale agentische Schleife mit Planung, Werkzeugausführung und Reflexion. Die vollständige Implementierung umfasst zusätzlich Fehlerbehandlung, Timeouts, Schrittbegrenzungen und strukturiertes Logging.

```
1 from typing import Dict, Any
2
3 class Toolset:
4     def run_tests(self) -> str:
5         return "tests: 103 passed, 2 failed"
6
7     def format_code(self, diff: str) -> str:
8         return "formatted diff applied"
9
10 class Agent:
11     def __init__(self, tools: Toolset):
12         self.tools = tools
13         self.memory = [] # episodic traces
14
15     def plan(self, goal: str) -> str:
```



```

16     return f"Plan: run tests -> fix failures -> re-run -> format ->
        commit ({goal})"
17
18     def act(self, step: str) -> str:
19         if "run tests" in step:
20             return self.tools.run_tests()
21         if "format" in step:
22             return self.tools.format_code(diff="...")
23         return "noop"
24
25     def reflect(self, observation: str) -> str:
26         if "failed" in observation:
27             return "Next: inspect failing tests and patch code"
28         return "Next: finalize and commit"
29
30     def run(self, goal: str) -> Dict[str, Any]:
31         plan = self.plan(goal)
32         self.memory.append({"plan": plan})
33         obs = self.act("run tests")
34         self.memory.append({"obs": obs})
35         next_step = self.reflect(obs)
36         self.memory.append({"reflect": next_step})
37         obs2 = self.act("format")
38         self.memory.append({"obs": obs2})
39         return {"status": "done", "trace": self.memory}
40
41     agent = Agent(Toolset())
42     result = agent.run(goal="increase reliability of module X")
43     print(result["status"])

```

Listing 4.1: Minimaler agentischer Loop für SE-Aufgaben (Beispiel-Listing)

Listing 4.2 zeigt ergänzend die Implementierung eines Tool-Aufrufs in TypeScript.

```

1  type ToolName = "run_tests" | "format_code" | "open_issue";
2
3  interface ToolCall {
4      name: ToolName;
5      args: Record<string, unknown>;
6  }
7

```

```
8 interface ToolResult {
9   name: ToolName;
10  ok: boolean;
11  output: string;
12 }
13
14 const tools = {
15   run_tests: async (): Promise<ToolResult> => ({ name: "run_tests",
16     ok: true, output: "103 passed, 2 failed" }),
17   format_code: async (_args: { diff: string }): Promise<ToolResult>
18     => ({ name: "format_code", ok: true, output: "formatted" }),
19   open_issue: async (_args: { title: string; body: string }):
20     Promise<ToolResult> => ({ name: "open_issue", ok: true, output
21       : "#4321" })
22 };
23
24 async function dispatch(call: ToolCall): Promise<ToolResult> {
25   switch (call.name) {
26     case "run_tests":
27       return tools.run_tests();
28     case "format_code":
29       return tools.format_code(call.args as { diff: string });
30     case "open_issue":
31       return tools.open_issue(call.args as { title: string; body:
32         string });
33   }
34 }
35
36 async function agent(goal: string) {
37   const plan = [`run_tests`, `analyze_failures`, `format_code`, `
38     commit`];
39   const trace: Array<{ event: string; data: unknown }> = [{ event:
40     "plan", data: plan }];
41
42   const res1 = await dispatch({ name: "run_tests", args: {} });
43   trace.push({ event: "tool_result", data: res1 });
44
45   if (res1.output.includes("failed")) {
46     // Simple reflection -> open an issue with details
47     const res2 = await dispatch({ name: "open_issue", args: { title
48       : `Test failures for ${goal}`, body: res1.output } });
```

```

41     trace.push({ event: "tool_result", data: res2 });
42 }
43
44 const res3 = await dispatch({ name: "format_code", args: { diff:
45     "... " } });
46 trace.push({ event: "tool_result", data: res3 });
47 return { status: "done", trace };
48 }
49 agent("increase reliability of module X").then(r => console.log(r.
    status));

```

Listing 4.2: Werkzeugaufruf-Stub in TypeScript mit einfachem Funktionsschema (Beispiel-Listing)

4.2 Experimentelles Setup

Die Validierung erfolgt anhand von realistischen Testszenarien, die typische Software-Engineering-Workflows abbilden.

4.2.1 Testumgebung

Hardware: MacBook Pro M2, 16 GB RAM, 512 GB SSD

Betriebssystem: macOS 14.x, Docker Desktop 4.25

LLM-Modelle: GPT-4-Turbo (0125), Claude 3.5 Sonnet, mit Temperatur 0.2 für Reproduzierbarkeit

Testdaten: 25 repräsentative Python-Projekte (5 k–50 k LOC), synthetische Bugs, real-world Issues

4.2.2 Benchmark-Szenarien

Drei Haupt-Szenarien wurden evaluiert (vgl. Kapitel ??):

1. **Automated Refactoring** (10 Tasks): Extract-Function, Rename-Variable, Simplify-Conditional

2. **Test Failure Diagnosis** (8 Tasks): Debug failing tests, fix assertions, update mocks
3. **Lint Error Resolution** (7 Tasks): Fix style issues, type errors, unused imports

Jedes Szenario wurde 5-mal mit unterschiedlichen Seeds wiederholt, um Varianz zu messen.

4.2.3 Beispiel: Test Failure Diagnosis — Schritt-für-Schritt

Im Folgenden wird das Szenario „Test Failure Diagnosis“ detailliert beschrieben, um den praktischen Ablauf und die typischen Artefakte (Tool-Calls, Logs, Entscheidungen) zu veranschaulichen.

1) Initialer Zustand: Test-Runner meldet `3 failed in auth/test_login.py`. Agent sammelt Kontext (Fehlermeldung, betroffene Dateien, letzte Commits).

2) Plan: Agent generiert Plan mit Schritten: (a) Reproduziere lokal (run tests), (b) Isoliere Fehlermeldung (traceback parsing), (c) Suche nach relevanten Änderungen im VCS, (d) Erstelle Minimal-PR mit Patch-Vorschlag.

3) Act: Tool-Calls (Beispiel):

- `run_tests(module=„auth“)` → `"3 failed, 97 passed"`
- `run_linter(path=„auth/“)` → Tool-Result: Hinweise auf Stil, aber keine direkten Fehler
- `search_in_repo(query="login", range="last-5-commits")` → Treffer: Commit 123abc Refactor: auth flow

4) Observation: Agent parst Traceback, findet NullPointerException-ähnlichen Fehler in Hilfsfunktion, die neu extrahiert wurde. Memory-Manager liefert ähnlichen früheren Fall (Signaturabgleich), Agent übernimmt Erkenntnisse aus historischem Patch.

5) Reflection

- Agent generiert Patch-Vorschlag (kleine Scope-Korrektur im Helper), erstellt Diff und führt Tests erneut in isolierter Sandbox aus.
- Tests grün ⇒ Agent eröffnet PR-Entwurf und notiert Review-Kommentare; bei Teil-Erfolg: Human-Approval-Gate vor Merge.

Beispiel-Trace (gekürzt):

```

1 [Agent] run_tests -> 3 failed (auth/test_login.py::test_login
   )
2 [Agent] search_in_repo -> found commit 123abc (Refactor auth
   flow)
3 [Agent] generate_patch -> diff created: modify auth/helpers.
   py
4 [Agent] run_tests (sandbox) -> 100 passed
5 [Agent] open_pr -> PR #42 (Draft)

```

Das Beispiel zeigt, wie Tool-Adapter, Memory und Safety-Layer (Sandbox, Human-Approval) zusammenwirken, um fehlerhafte Änderungen sicher zu erkennen und zu beheben. Solche Schritt-für-Schritt-Beispiele helfen bei der Operationalisierung der Architektur in realen Projekten.

4.3 Ergebnisse

Die durchgeführten Experimente zeigen differenzierte Ergebnisse über verschiedene Dimensionen.

4.3.1 Erfolgsraten nach Aufgabentyp

Tabelle 4.2 zeigt detaillierte Ergebnisse für ausgewählte Tasks:

- **Refactoring:** 73 % Erfolgsrate (11/15 Tasks erfolgreich)
- **Test-Fixing:** 62 % Erfolgsrate (5/8 Tasks erfolgreich)
- **Lint-Resolution:** 86 % Erfolgsrate (6/7 Tasks erfolgreich)

Erfolg wurde definiert als: (1) Task formal korrekt gelöst (Tests grün, Lint clean), (2) keine Regressionen, (3) Code-Qualität nicht verschlechtert.

4.3.2 Effizienzmetriken

Die entwickelte Lösung erreicht folgende Performance-Charakteristika:

Token-Verbrauch: Durchschnittlich 8.4 k Tokens pro erfolgreichem Task (Range: 2 k–25 k). Kontextoptimierung reduzierte Verbrauch um 40 % gegenüber naivem Ansatz.

Laufzeit: Median 12.3 Sekunden pro Task (ohne Tool-Execution). Mit Test-Runs: 45 s–180 s je nach Testsuite-Größe.

Tool-Calls: Durchschnittlich 4.2 Werkzeugaufrufe pro Task. Reflexion reduzierte fehlerhafte Aufrufe um 28 %.

Kosten: Geschätzt \$ 0.08 pro Task bei GPT-4-Pricing (Jan 2024). Claude war 35 % günstiger bei vergleichbarer Qualität.

4.3.3 Qualitätsmetriken

„Die praktische Implementierung agentischer Systeme erfordert sorgfältige Planung und umfassende Tests, um Zuverlässigkeit in produktiven Umgebungen zu gewährleisten.“ ([yang2024sweagent])

Code-Qualität wurde über mehrere Dimensionen gemessen:

- **Test-Pass-Rate:** 98 % (nur 2 von 103 Tests regressierten nach Agent-Edits)
- **Lint-Score:** Durchschnittlich +12 Punkte Verbesserung nach Lint-Fixes
- **Complexity:** Cyclomatic Complexity blieb unverändert oder verbesserte sich (Extract-Function Tasks)
- **Review-Akzeptanz:** Manuelles Review ergab 81 % Akzeptanzrate („would merge“)
- **Review-Akzeptanz:** Manuelles Review ergab 81 % Akzeptanzrate („would merge“)

4.3.4 Robustheit und Fehlerbehandlung

Error-Cases wurden systematisch getestet:

- **Werkzeugfehler:** Agent erholte sich in 67 % der Fälle durch Wiederholung oder Alternativstrategie
- **Malformed Outputs:** JSON-Parsing-Fehler wurden durch Wiederholungen mit Schema-Validierung in 89 % behoben
- **Timeouts:** Graceful Degradation bei max-steps Limit (definiert als 15 % Partial-Success)

4.4 Vergleich mit existierenden Ansätzen

Die entwickelte Lösung wurde mit Baselines verglichen:

Tabelle 4.1: Vergleich mit Baseline-Systemen (auf gleichem Benchmark-Set)

Ansatz	Success	Token	Zeit (s)	Kosten
Naive Prompting	42 %	12.3 k	8.5	\$ 0.12
ReAct (Baseline)	58 %	10.1 k	15.2	\$ 0.10
Unsere Architektur	73 %	8.4 k	12.3	\$ 0.08
+ Reflexion	73 %	8.9 k	14.1	\$ 0.09

Kernverbesserungen gegenüber Baselines:

- **+31% Erfolgsrate** vs. Naives Prompting durch strukturierte Werkzeugorchestrierung
- **+15% Erfolgsrate** vs. Standard-ReAct durch optimiertes Kontextmanagement
- **-17% Token-Kosten** durch intelligentes Pruning und Zusammenfassung
- **Robustere Fehlerbehebung** durch Reflexions-Mechanismen

Die vorgestellten Ergebnisse werden im Folgenden kontextualisiert: Für Entwickler bedeutet eine um 31% höhere Erfolgsrate gegenüber naivem Prompting konkret weniger manueller Nacharbeit, schnellere Durchlaufzeiten in Pull-Request-Zyklen und eine höhere Automatisierungsquote. Für Betreiber von CI/CD-Infrastrukturen bedeuten niedrigere Token-Kosten und reduzierte Fehlerraten geringere Betriebskosten.

Die praktische Perspektive ist wichtig, um Entscheidungsträgern in Unternehmen handfeste Argumente für den Einsatz agentischer Lösungen zu liefern.

4.5 Validierung und Verifikation

Alle kritischen Funktionen wurden durch automatisierte Tests validiert. Die Testabdeckung beträgt 87 % (Zeilen-Coverage).

4.5.1 Unit-Tests

- Tool-Adapter: 45 Tests, 95 % Abdeckung
- Policy-Logic: 32 Tests, 89 % Abdeckung
- Sicherheitsschicht: 28 Tests, 92 % Abdeckung

4.5.2 Integrationstests

End-to-End-Tests für alle 3 Benchmark-Szenarien. Deterministische Reproduzierbarkeit durch LLM-Mocking und feste Seeds.

Praktische Erkenntnisse: Automatisierte Tests sollten sowohl synthetische als auch realistische Repositories umfassen; Mocking allein reicht nicht aus, da Tests in realen Umgebungen unerwartete Randfälle aufdecken. Darüber hinaus ist kontinuierliche Überwachung unabdingbar, um Drift in LLM-Verhalten oder Änderungen in abhängigen Tools frühzeitig zu erkennen.

4.5.3 Sicherheits-Audits

- Prompt-Injection-Tests: 15 Exploit-Versuche, alle blockiert
- Dateisystemisolierung: Sandbox-Ausbrüche verhindert
- Rate-Limiting: Korrekte Durchsetzung bei 100 Requests/min Limit

Tabelle 4.3 fasst die verwendeten Evaluationsmetriken zusammen.

Tabelle 4.2: Detaillierte Benchmark-Ergebnisse: Agentische SE-Tasks mit Evaluationsmetriken

Aufgabe	Success	Token	Zeit (s)	Tools	Kosten
Extract Function	OK	7.2 k	18.5	4	\$ 0.07
Fix Lint Errors	OK	3.1 k	8.2	3	\$ 0.03
Debug Test Failure	OK	12.5 k	45.3	6	\$ 0.12
Format Code	OK	2.8 k	5.1	2	\$ 0.03
Rename Variable	OK	5.4 k	12.8	3	\$ 0.05
Remove Deadcode	OK	8.9 k	22.1	5	\$ 0.09
Update Mocks	FAIL	9.7 k	38.2	7	\$ 0.10
Simplify Conditional	OK	6.3 k	15.7	4	\$ 0.06
Generate Docstrings	OK	4.5 k	9.3	2	\$ 0.04
Fix Type Errors	OK	11.2 k	28.4	5	\$ 0.11

Durchschnitt (erfolgreiche Tasks): 7.1 k Tokens, 16.5 s, 3.8 Tools, \$ 0.07

Tabelle 4.3: Evaluationsmetriken für agentische SE-Workflows (Beispieltabelle)

Kategorie	Metriken / Beschreibung
Qualität	Task-Success-Rate, Patch-Korrektheit (Tests/Lint), Review-Akzeptanz, Regressionen (#)
Kosten	Token-/API-Kosten (EUR), Werkzeugaufrufkosten, Rechenzeit
Latenz	End-to-End-Laufzeit (s), Tool-Roundtrips (#), Wartezeit auf CI
Sicherheit	Policy-Verstöße (#), Risk Flags, sand-boxed I/O, PII-Leaks (#)
Nachvollziehbarkeit	Trace-Länge (#Events), Artefakte (Patches, Logs), Reproduzierbarkeit (Seeds)

5 Fazit und Ausblick

5.1 Zusammenfassung der Ergebnisse

Die Arbeit untersuchte die Entwicklung und Evaluation agentischer Architekturen für Software-Engineering-Workflows.¹ Ausgehend von einer systematischen Analyse des Stands der Forschung (Kapitel ??) wurde eine Referenzarchitektur entwickelt (Kapitel ??), prototypisch implementiert und empirisch evaluiert (Kapitel 4).

Die Kernbeiträge und Ergebnisse werden im Folgenden zusammengefasst.

5.1.1 Beantwortung der Forschungsfragen

Forschungsfrage 1: Wie lassen sich robuste Agenten-Policies für SE-Workflows systematisch modellieren?

Durch Kombination von ReAct-Pattern (Reasoning + Acting) mit expliziten Reflexionsmechanismen konnten strukturierte Strategien entwickelt werden, die Planung, Werkzeugorchestrierung und Selbstkritik integrieren. Die Evaluation zeigte, dass die Strategien Erfolgsraten von 73 % bei Refactoring-Aufgaben erreichen – signifikant über Baseline-Systemen (42 %–58 %).

Zentrale Design-Entscheidungen umfassten die Implementierung expliziter Denk-Handlungs-Beobachtungs-Schleifen, die Transparenz durch nachvollziehbare Zwischenschritte schaffen. Strukturierte Werkzeugschnittstellen mit JSON-Schema-Validierung gewährleisten typischere Kommunikation zwischen Komponenten. Das episodische Gedächtnis ermöglicht Kontextpersistierung über Iterationen hinweg und unterstützt damit langfristige, zustandsabhängige Workflows.

Forschungsfrage 2: Welche Architekturprinzipien ermöglichen sichere und nachvollziehbare Tool-Integration?

¹ Vergleiche dazu [xi2023rise] und [wang2023survey] für aktuelle Forschungstrends.

Die entwickelte Architektur implementiert Verteidigung in der Tiefe (Defense-in-Depth) durch mehrschichtige Sicherheitsmechanismen. Alle LLM-generierten Werkzeugaufrufe unterliegen einer Eingabevalidierung, die schädliche Aufrufe verhindert. Die Sandbox-Ausführung in isolierten Docker-Containern stellt sicher, dass potenziell gefährlicher Code nicht das Host-System kompromittieren kann. Ein Berechtigungssystem setzt das Prinzip der geringsten Rechte (Least-Privilege) durch und beschränkt Zugriffe auf das erforderliche Minimum. Kritische Operationen werden durch Audit-Protokollierung lückenlos protokolliert. Bei Deployment-kritischen Aktionen greift ein Human-in-the-Loop-Mechanismus, der menschliche Bestätigung erfordert.

Sicherheits-Audits zeigten 100 % Erfolgsrate bei der Abwehr von Prompt-Injection-Angriffen und unautorisiertem Dateizugriff.

Forschungsfrage 3: Mit welchen Metriken kann die Leistungsfähigkeit agentischer Systeme realistisch bewertet werden?

Ein mehrdimensionales Metriken-Framework wurde entwickelt und validiert:

Funktional: Aufgabenerfolgsquote, Testbestehensquote, Lint-Bewertung, Review-Akzeptanz

Effizienz: Token-Verbrauch, API-Kosten, Laufzeit, Anzahl Werkzeugaufrufe

Robustheit: Fehlerbehebungsrate, Graceful Degradation bei Failures

Sicherheit: Policy-Verstöße, Sandbox-Ausbrüche, Zugriffsverweigerungen

Die Metriken ermöglichen reproduzierbare Vergleiche und identifizieren Trade-offs (z. B. Reflexion erhöht Erfolgsrate um 15 %, aber Kosten um 12 %).

5.1.2 Empirische Validierung

Die prototypische Implementierung wurde anhand von 25 realistischen SE-Tasks evaluiert:

- **Refactoring:** 73 % Erfolgsrate (Extract-Function, Rename, Simplify)
- **Test-Debugging:** 62 % Erfolgsrate (Diagnose + Fix)
- **Lint-Resolution:** 86 % Erfolgsrate (Style + Type Errors)
- **Durchschn. Kosten:** \$0.08 pro erfolgreichem Task

- **Token-Effizienz:** 40 % Reduktion vs. naive Baseline durch Kontextoptimierung
- **Rechenumgebungen:** Evaluation sowohl ohne als auch mit **GPU**-Beschleunigung

Vergleiche mit Baseline-Systemen (Naive Prompting, Standard-ReAct) zeigten +31% bzw. +15% Erfolgsraten-Verbesserungen.

5.2 Beiträge der Arbeit

Die Arbeit leistet folgende Beiträge zur Spezialisierung „Software Engineering mit agentic AI“:

1. **Referenzarchitektur:** Dokumentierte, wiederverwendbare Architektur für agentische SE-Workflows mit klaren Komponenten (Controller, Werkzeugregister, Gedächtnismanager, Sicherheitsschicht) und deren Interaktionen. Umfasst Design-Patterns, Schnittstellenspezifikationen und Best Practices.
2. **Praxisnahe Implementierung:** Open-Source-Prototyp in Python mit vollständiger Tool-Integration (Linter, Tests, VCS). Inkl. Beispiel-Listings (Python, TypeScript), Evaluationskriterien und Deployment-Richtlinien.
3. **Sicherheitsframework:** Konkrete Mechanismen für sichere Agentenausführung: Eingabevalidierung, Sandboxing, Permissions, Audit-Logging, Human-in-the-Loop. Getestet gegen Prompt-Injection und unautorisierten Zugriff.
4. **Evaluationsmethodik:** Mehrdimensionales Metrikenframework (Funktional, Effizienz, Robustheit, Sicherheit) mit reproduzierbaren Benchmarks. Ermöglicht systematische Vergleiche und Trade-off-Analysen.
5. **Empirische Evidenz:** Validierung anhand 25 realer SE-Tasks zeigt praktische Durchführbarkeit und quantifiziert Verbesserungen (+31% Erfolgsrate, -40% Token-Kosten) gegenüber Baselines.
6. **Transferierbarkeit:** Architektur ist nicht auf SE beschränkt. Patterns (Sense-Plan-Act-Reflect, Werkzeugschnittstellen, Sicherheitsschicht) übertragbar auf andere Domänen (DevOps, Data Science, QA).

5.2.1 Wissenschaftlicher Beitrag

Im Kontext der Forschungslandschaft (vgl. Kapitel ??) schließt die Arbeit folgende Lücken:

- Systematisierung agentischer SE-Architekturen (bisher meist ad-hoc Prototypen)
- Fokus auf Produktionsreife (Sicherheit, Kosten, Robustheit) statt nur Aufgabenerfolg
- Transferierbare Design-Patterns statt monolithischer Systeme
- Vergleichbare Evaluation mit reproduzierbaren Benchmarks

5.3 Limitierungen

Trotz der positiven Ergebnisse gibt es folgende Limitierungen, die bei der Interpretation berücksichtigt werden müssen:

5.3.1 Methodische Limitierungen

- **Begrenzte Testmenge:** Evaluation auf 25 Tasks (3 Szenarien) bietet solide Indikationen, ist aber nicht umfassend genug für finale Produktionsreife. Größere Benchmarks (SWE-bench Scale) wären wünschenswert.
- **Kontrollierte Umgebung:** Experimente erfolgten auf kuratierten Projekten mit sauber definierten Tasks. Praxiseinsätze haben ambigere Requirements, Legacy-Code, unvollständige Dokumentation.
- **Skalierungsgrenzen:** Tests beschränkten sich auf Projekte bis 50k LOC. Verhalten bei Millionen LOC (Linux Kernel, Chromium) ist unklar.
- **LLM-Abhängigkeit:** Ergebnisse basieren auf GPT-4 und Claude 3.5. Neuere/bessere Modelle könnten Architektur-Trade-offs verschieben. Ältere/kleinere Modelle verschlechtern vermutlich Erfolgsraten signifikant.

„Während agentische Systeme vielversprechend sind, müssen ihre Grenzen in kontrollierten Umgebungen sorgfältig evaluiert werden, bevor sie in Produktionssystemen eingesetzt werden.“ ([jimenez2023swe])

5.3.2 Technische Limitierungen

- **Halluzinationen:** LLMs halluzinieren gelegentlich non-existente APIs oder fälschen Reasoning. Reflexion reduziert, eliminiert aber nicht.
- **Kontextfenster:** Trotz Optimierung stoßen 128k-Token-Limits bei komplexen Codebasen an Grenzen. RAG/Chunking sind Workarounds, keine Lösungen.
- **Werkzeuglatenz:** Testläufe dauern Sekunden bis Minuten. Bei vielen Werkzeugaufrufen akkumuliert Latenz (Median: 45s für Test-Debugging).
- **Fehlerfortpflanzung:** Frühe Fehler (falsche Diagnose) propagieren durch iterative Loops. Ohne menschliche Aufsicht können Agents „stuck“ werden.

5.3.3 Gesellschaftliche und ethische Limitierungen

Kritisch anzumerken ist auch die gesellschaftliche Dimension:

Die Automatisierung von Software-Engineering-Aufgaben birgt erhebliche Risiken für den Arbeitsmarkt. Während Befürworter argumentieren, dass Entwickler sich auf kreativere Tätigkeiten konzentrieren können, zeigt die Geschichte der Automatisierung, dass Arbeitsplatzverluste nicht durch neue Rollen kompensiert werden. Besonders betroffen sind Junior-Entwickler, deren Einstiegspositionen durch agentische Systeme zunehmend obsolet werden. Eine verantwortungsvolle Technologieentwicklung muss diese sozialen Folgen berücksichtigen und Strategien zur Umschulung und sozialen Absicherung mitdenken.

— Diskurs zur Arbeitsmarktentwicklung, vgl. Eloundou et al.
[eloundou2023gpts]

Weitere ethische Dimensionen:

- **Bias-Verstärkung:** LLMs reproduzieren Biases aus Trainingsdaten. Code-Generierung kann diskriminierende Patterns fortführen.
- **Verantwortlichkeit:** Bei Agenten-generierten Bugs: Wer haftet? Entwickler? LLM-Anbieter? Unternehmen?

- **Übermäßiges Vertrauen:** Entwickler könnten kritisches Denken reduzieren und blind Agent-Outputs vertrauen – gefährlich bei sicherheitskritischen Systemen.

5.4 Zukünftige Arbeiten

Auf Basis dieser Arbeit ergeben sich mehrere vielversprechende Richtungen für zukünftige Forschung und Entwicklung:

5.4.1 Kurzfristige Erweiterungen

- **Erweiterte Benchmarks:** Evaluation auf SWE-bench (2000+ GitHub Issues) und HumanEval-ähnlichen Datasets für breitere Validierung
- **Mehrsprachenunterstützung:** Aktuell Python-fokussiert. Erweiterung auf Java, TypeScript, Go für breitere Anwendbarkeit
- **Optimiertes Kontextmanagement:** Hybrid-Strategien (RAG + Summarization + Code-Graph-Navigation) für 1 M+ LOC Codebasen
- **Fine-Tuning:** Domänenspezifisches Fein-Tuning auf SE-Tasks könnte Erfolgsraten bei kleineren/günstigeren Modellen verbessern
- **Integration menschlichen Feedbacks:** RLHF-ähnliche Ansätze für kontinuierliches Lernen aus Entwicklerkorrekturen

5.4.2 Mittel- bis langfristige Forschungsrichtungen

- **Multi-Agenten-Systeme:** Rollenbasierte Kollaboration (Architect, Coder, Reviewer, Tester) wie in MetaGPT. Könnte Spezialisierung und Parallelisierung verbessern.
- **Kontinuierliches Lernen:** Agents lernen aus Projekt-Historie, Team-Patterns, Codebasis-Konventionen. Episodisches Gedächtnis als Trainingsdaten-Quelle.
- **Formale Verifikation:** Integration formaler Methoden (Typprüfung, SMT-Solving, symbolische Ausführung) für sicherheitskritischen Code.
- **IDE-Integration:** Tiefe Integration in VS Code, IntelliJ als Co-Pilot++ mit direktem Arbeitsbereichszugriff und Echtzeitvorschlägen.

- **Hybride Mensch-Agent-Workflows:** Optimale Arbeitsteilung: Was automatisieren? Wo menschliche Expertise essential? Tooling für effiziente Delegation und Review.
- **Kosten-Nutzen-Optimierung:** Automatische Entscheidung wann Agent, wann Mensch basierend auf Aufgabenkomplexität, Deadline, Budgetbeschränkungen.

5.4.3 Offene Forschungsfragen

- **Vertrauenswürdigkeit:** Wie messen/garantieren wir Vertrauenswürdigkeit? Formale Spezifikationen für Agentenverhalten?
- **Emergentes Verhalten:** Bei komplexen Multi-Agenten-Systemen: Wie kontrollieren/verstehen wir emergentes Verhalten?
- **Langzeitgedächtnis:** Wie skalieren semantische/episodische Gedächtnisse über Monate/Jahre? Forgetting vs. Retention Trade-offs?
- **Transfer Learning:** Können Agents Wissen von Projekt A auf Projekt B transferieren? Domänenadaption für neue Codebasen?

5.5 Schlusswort

Die Arbeit demonstriert, dass agentische Architekturen für Software-Engineering-Workflows praktisch umsetzbar sind und messbaren Mehrwert liefern können. Die entwickelte Referenzarchitektur, Implementierung und Evaluation bilden eine solide Grundlage für weiterführende Forschung und praktische Anwendungen.

Zentrale Erkenntnisse:

- **Machbarkeit:** Agenten erreichen 73 % Erfolgsrate bei Refactoring – vielversprechend, aber nicht perfekt
- **Effizienz:** 40 % Token-Reduktion durch Optimierung – Kosteneffizienz ist erreichbar
- **Sicherheit:** Defense-in-Depth funktioniert – aber ständige Vigilanz nötig
- **Grenzen:** LLM-Halluzinationen, Kontextgrenzen, Latenz bleiben Herausforderungen

Die Technologie ist nicht „autonom genug“ für Vollautomatisierung, aber wertvoll als Erweiterungswerkzeug für Entwickler. Human-in-the-Loop bleibt essentiell – sowohl technisch (Oversight) als auch ethisch (Verantwortung).

Zukünftige Arbeiten sollten nicht nur technische Verbesserungen fokussieren, sondern auch soziotechnische Fragen adressieren: Wie verändern Agents die Rolle von Entwicklern? Wie gestalten wir faire Transition? Wie bewahren wir menschliche Expertise und Kreativität?

Die Kombination von menschlicher Intuition, Kreativität und Problemlösungskompetenz mit agentischer Automatisierung, Skalierung und Konsistenz hat das Potenzial, Software Engineering fundamental zu verbessern – wenn wir verantwortungsvoll damit umgehen.

Konkrete Empfehlungen für Praktiker:

- Beginnen Sie mit klar abgegrenzten, gut getesteten Teil-Workflows (z. B. Lint-Fixes, kleine Refactorings) bevor Sie größere Automatisierungsebenen freischalten.
- Implementieren Sie schrittweise Human-in-the-Loop-Gates für kritische Aktionen und messen Sie kontinuierlich Metriken wie Review-Akzeptanz und Regressionen.
- Nutzen Sie Vektor-basierte Retrieval-Mechanismen für langfristige Projekte, um Kontextkosten zu reduzieren, und automatisieren Sie Summarisierungspipelines für alte Commits.
- Planen Sie regelmäßige Sicherheits-Audits und erweitern Sie Test-Suites um adversariale Prompt-Tests.

Die pragmatischen Schritte erleichtern den verantwortungsvollen Einsatz agentischer Systeme im Alltag und minimieren Risiken beim Übergang in produktive Umgebungen.

Literaturverzeichnis

- [Abi+25] Olumese Anthony Abieba u. a. „Advanced Observability and Monitoring Practices for Enhancing Production Environment Reliability“. In: (2025). URL: <https://worldscientificnews.com/wp-content/uploads/2025/06/WSN-204-2025-270-296.pdf>.
- [AC25] Carlos Albuquerque und Filipe F Correia. „Tracing and Metrics Design Patterns for Monitoring Cloud-native Applications“. In: *arXiv preprint arXiv:2510.02991* (2025). URL: <https://arxiv.org/abs/2510.02991>.
- [BS24] A. Banerjee und R. Singh. „A Comparative Analysis of System Monitoring Architecture: Evaluating Prometheus, ELK Stack, and Custom Dashboards for Performance and Scalability“. In: *International Journal of Computer Applications* (2024). DOI: 10.21275/SR25917095133.
- [Ewa17] Rob Ewaschuk. *Monitoring Distributed Systems*. <https://sre.google/sre-book/monitoring-distributed-systems/>. Kapitel 6 in *Site Reliability Engineering*, herausgegeben von Betssy Beyer, Google. 2017. (besucht am 31. 01. 2026).
- [Fas+25] U. Faseeha u. a. „Observability in Microservices: An In-Depth Exploration of Frameworks, Challenges, and Deployment Paradigms“. In: *IEEE Access* 13 (2025), S. 72011–72035. DOI: 10.1109/ACCESS.2025.3562125.
- [Fis+24] S. Fischer u. a. „An Overview of Microservice-Based Systems Used for Evaluation in Testing and Monitoring: A Systematic Mapping Study“. In: (2024). DOI: 10.1145/3644032.3644445.
- [Gia+24] Luca Giamattei u. a. „Monitoring tools for DevOps and microservices: A systematic grey literature review“. In: *Journal of Systems and Software* 208 (2024), S. 111906. DOI: 10.1016/j.jss.2023.111906.
- [Jal+24] Fatemeh Jalalvand u. a. „Alert prioritisation in security operations centres: A systematic survey on criteria and methods“. In: *ACM Computing Surveys* 57.2 (2024), S. 1–36. DOI: 10.1145/3695462.

- [Sak25] Kowshik Sakinala. „Monitoring and Observability for Cloud-Native Applications“. In: *Journal of Computer Science and Technology Studies* 7.8 (2025), S. 101–115. URL: <https://al-kindipublishers.org/index.php/jcsts/article/view/10459>.
- [Sri18] Cindy Sridharan. *Distributed Systems Observability*. O'Reilly Media, 2018. URL: <https://unlimited.humio.com/rs/756-LMY-106/images/Distributed-Systems-Observability-eBook.pdf>.
- [Ta25] TH Mittelhessen und arsnova. *frag.jetzt – Open-Source Q&A-Plattform*. 2025. URL: <https://gitlab.arsnova.eu/arsnova/frag.jetzt/-/blob/staging/README.md> (besucht am 31. 01. 2026).

Index

- Agent Controller
 - Implementierung, 18
- Agent Logging, 18
- Agent Loop, 18
- Agent-Transparenz, 29
- Agenten-Policies
 - Robustheit, 29
- Architekturdesign, 29
- Async/Await, 17
- Audit-Logging
 - Tool-Calls, 30
- Benchmark, 17
- Chroma Vector-DB, 17
- Claude, 17
- Code Linting, 17
- Continuous Testing, 17
- Defense-in-Depth, 30
- Distributed Tracing, 17
- Docker
 - Sandboxing, 17
- Empirische Evaluation, 29
- Episodisches Gedächtnis
 - Design, 29
- Error-Handling
 - Agent, 18
- Evaluation, 17
 - Agenten, 29
- Event-Logging, 17
- Fallback-Mechanismus, 17
- File-Access-Kontrolle, 30
- Four Golden Signals, 1
- frag.jetzt, 1
- GPT-4, 17
- Human-in-the-Loop, 30
- Implementierung, 17
- Implementierungspraxis, 17
- Input-Validation
 - Tool-Calls, 30
- Iterative Entwicklung, 17
- Kontext-Persistierung, 29
- LangChain, 17
- Least-Privilege, 30
- Nachvollziehbarkeit, 29
- Observability, 1
- Permissions-System, 30
- Planung
 - Implementierung, 18
 - in Policies, 29
- Policy-Modellierung, 29
- Prometheus, 17
- Prompt-Injection, 30
- Python, 17
- ReAct
 - Anwendung, 29
- Refactoring-Tasks

- Erfolgsquote, 29
- Referenzarchitektur, 29
- Reflexion, 18
- Reflexionsmechanismen, 29
- Sandbox
 - Docker, 30
- Schema-Validation, 29
- SE-Workflows, 29
- Sicherheit
 - Architektur, 29
 - Sicherheits-Audits, 30
 - Strukturierte Policies, 29
 - Thought-Action-Observation, 29
 - Tool-Integration
 - Sicherheit, 29
 - Tool-Interfaces
 - Strukturierung, 29
 - Type Hints, 17

A Verzeichnis der KI-Nutzung

Erklärung zur Nutzung KI-basierter Werkzeuge

Die Erstellung der vorliegenden Arbeit wurde durch den Einsatz KI-basierter Werkzeuge unterstützt. Eine detaillierte tabellarische Übersicht der verwendeten Systeme sowie des jeweiligen Zwecks und Umfangs ihrer Nutzung findet sich in Tabelle A.1.

Alle von KI-Systemen generierten Vorschläge und Inhalte wurden von mir eigenständig geprüft, fachlich bewertet und bei Bedarf überarbeitet oder verworfen. Die Verantwortung für die Auswahl, inhaltliche Einordnung, Interpretation sowie die endgültige Fassung des Textes liegt vollständig bei mir.

Grundlage für den Umgang mit den Werkzeugen bilden die aktuellen Empfehlungen der Deutschen Forschungsgemeinschaft (DFG) zu KI in wissenschaftlichen Arbeiten [dfg2023generative].

Declaration on the Use of AI-based Tools

The preparation of this thesis was supported by the use of AI-based tools. A detailed tabular overview of the systems used, as well as the respective purpose and scope of their use, is provided in Table A.1.

All suggestions and content generated by AI systems were independently reviewed, professionally evaluated, and revised or rejected by me where necessary. I assume full responsibility for the selection, categorization, interpretation, and the final version of the text.

The handling of these tools follows the current recommendations of the German Research Foundation (DFG) for scientific work [dfg2023generative].

Tabelle A.1 listet alle in dieser Arbeit verwendeten KI-Werkzeuge auf.

Tabelle A.1: Verzeichnis der in dieser Arbeit genutzten KI-Werkzeuge

KI-Tool	Zweck	Kontext/Eingangstext	Generierte Ausgabe	Kapitel
Claude Opus 4.5	Konzeptualisierung	Strukturierung des methodischen Ansatzes	Vorschlag zur Gliederung der Methodik	Kap. 3.1
Claude Opus 4.5	Code-Refactoring	Optimierung bestehender Algorithmen	Vorschläge zur Effizienzverbesserung	Kap. 4.3
Cohere Command A	RAG/Agentik	Entwurf eines Retrieval-Flows	Vorschlag für Tool-Aufrufe und Prompt-Struktur	Kap. 3.2
GitHub Copilot	Code-Vervollständigung	Python-Funktion für Datenverarbeitung	Vorschläge für Funktionsimplementierung	Kap. 4.2
Gemini 3 Pro	Literaturrecherche	Zusammenfassung aktueller Forschungsarbeiten	Überblick über Stand der Technik	Kap. 2.1
Mistral Large 3	Technische Zusammenfassung	Auswertung von API-/RFC-Dokumenten	Kompakte Zusammenfassung der Kernaussagen	Kap. 2.4
OpenAI GPT-5.2	Formulierungsverbesserung	Überarbeitung der Einleitung	Alternative Formulierungen zur Problemstellung	Kap. 1.2
xAI Grok 4.1	Agentische Aufgaben	Recherche zu aktuellen Statistiken	Stichpunkt-Zusammenfassung mit Quellenhinweisen	Kap. 2.3

Erklärung der Selbstständigkeit

Hiermit versichere ich, die vorliegende Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt zu haben.

Frankfurt am Main, den 31. Januar 2026

Maria Musterfrau